**Technik der MMI**, UXD-B
Praktikum, WiSe 2026/27

**Praktikumsblatt 4**
Prof. Dr. Simon Nestler, Prof. Dr. Andreas Riener

# Arduino: Ultraschall, IR und Displays

**Worum geht es in diesem Blatt?**

Sie erweitern den Arduino um Ausgabe und Abstandsmessung. Sie geben Text und Grafik auf einem OLED-Display aus, binden Infrarotsensoren an, probieren weitere Displaytypen aus und messen mit Ultraschall Entfernungen. Die letzte Aufgabe ist bewusst offen gehalten: Dort kombinieren Sie die Bausteine zu etwas Eigenem.

**Was erwarten wir von der Dokumentation (Protokoll)?**

- Beantworten Sie die gestellten Fragen in einem Protokoll (Moodle-Fragebogen/Quiz)
- Machen Sie zusätzlich Screenshots Ihrer Programme und Fotos, die Ihre Lösungen in „Aktion“ zeigen; vergessen Sie nicht, die Programme zu speichern!
- Halten Sie Ihre eigene Interpretation fest: Was haben Sie im Experiment erkannt, was ist Ihnen aufgefallen?
- Welche Probleme sind Ihnen begegnet, und wie haben Sie sie gelöst?

**Wie?**

- **Bearbeiten Sie zur Abgabe den Test „Abgabe Praktikumsblatt 4“** in Moodle (Kursraum Technik der Mensch-Maschine Interaktion).
- **Kopieren Sie Ihre Erläuterungen und Ihren Quellcode in die jeweiligen Antwortfelder. Hängen Sie Screenshots und Bilder an, denn direkt ins Moodle-Quiz einfügen lassen sie sich in einigen Browsern nicht.**

**Abgabe?**

- **Abgabe innerhalb von 48 Stunden** nach Ihrem Praktikumstermin, in Moodle

Der große Vorteil der Arduino-Plattform: Für sehr viele Hardware-Komponenten gibt es bereits fertige Tutorials und/oder Programme. Das macht es viel leichter, weitere Sensoren hinzuzufügen. An verschiedenen Beispielen probieren Sie hier praktisch aus, wie Sie zusätzliche Sensoren finden und mit externen Bibliotheken in Ihre Programme einbinden.

## 1) Meine ersten Ausgaben auf einem OLED-Display

Im ersten Schritt schließen Sie ein externes Display an und geben Texte darauf aus

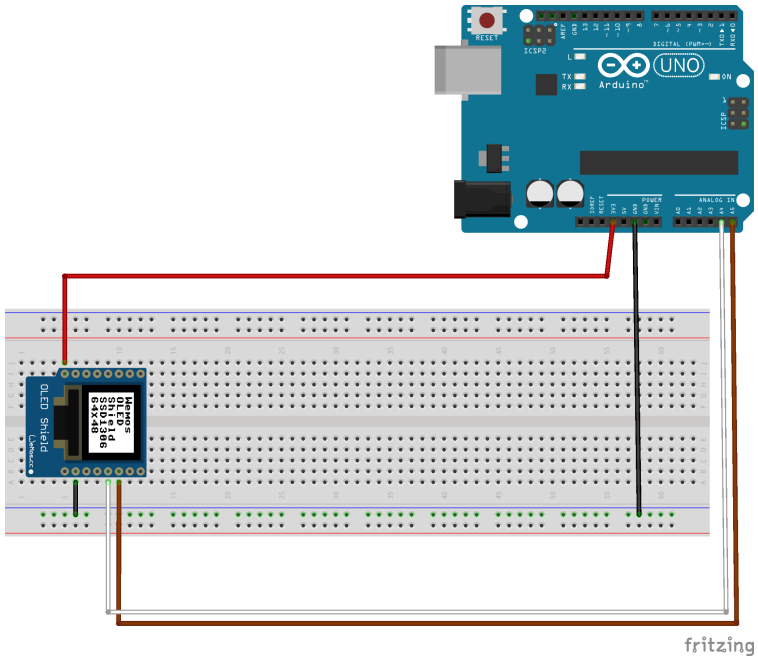
### Sie brauchen folgende Komponenten

- Arduino UNO-Board
- "*Breadboard*"
- OLED Display
- Kabel

### Vorbereitung

- Starten Sie am PC die Arduino-Entwicklungsumgebung
- Verbinden Sie das Arduino-Board per USB-Kabel mit dem PC
- Konfigurieren Sie die Arduino IDE wieder für den korrekten Chip (µC auswählen): Menü „Tools" → „Board" → „Arduino UNO R4 WIFI" auswählen
- Konfigurieren Sie die serielle Schnittstelle: Menü „Tools" → „Serial Port" → „COM1" (den richtigen Port auswählen; falls Sie ihn nicht kennen: siehe Windows-Systemsteuerung)
- Legen Sie über „File" → „New" einen neuen Sketch an

### Aufbau




- Schalten Sie den Arduino stromlos (USB-Kabel abstecken)

- Das Display **wird mit 3.3V betrieben**; verbinden Sie den linken unteren PIN des Displays (Beschriftung 3V3) mit dem 3V3 (oder VCC) Ausgang Ihres Arduino
- Verbinden Sie den zweiten PIN rechts unten des Displays (Beschriftung G) mit dem Ausgang GND des Arduinos. Mit Blick auf die nächsten Aufgaben empfiehlt es sich, den Ausgang des Arduino mit der Masse-Leiste des Breadboards zu verbinden und die Masse dann dort abzugreifen (wie in der Abbildung dargestellt).
- Verbinden Sie den Ausgang D2 (SDA) des Displays mit dem Eingang A4 des Arduino
- Verbinden Sie den Ausgang D1 (SCL) des Displays mit dem Eingang A5 des Arduino

Jetzt können Sie die erste Ausgabe auf dem Display anzeigen. Installieren Sie dafür über die Arduino IDE die beiden Adafruit Bibliotheken (Menü „Sketch“ → „Bibliothek einbinden“ → „Bibliotheken verwalten“ → „Adafruit GFX“ bzw. „Adafruit SSD1306“ suchen → „Install“). **Wichtig:** Geben Sie bei der Suche nach den Bibliotheken den Unterstrich im Paketnamen (z. B. in „Adafruit_GFX“) nicht mit ein.

```
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define OLED_RESET 0
Adafruit_SSD1306 display(OLED_RESET);

void setup()   {
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.display();

  display.setTextSize(1);
  display.setTextColor(WHITE);
}

void loop() {
  display.clearDisplay();
  display.setCursor(0, 0);

  display.println("TMI macht viel Spaß!");
  display.display();
  delay(50);
}
```

**Frage 1:** Funktioniert Ihr Programm? Wird der Text vollständig auf dem Display dargestellt? Wozu dient der Aufruf `display.setCursor(0, 0)`?

Der Treiber für einige unserer Displays ist ursprünglich für Displays mit 128 x 64 Pixeln ausgelegt. Unser Display hat aber nur 64 x 48 Pixel. Unter Umständen dürfen Sie Ihre Ausgabe deshalb nicht an der Position (0, 0) beginnen.

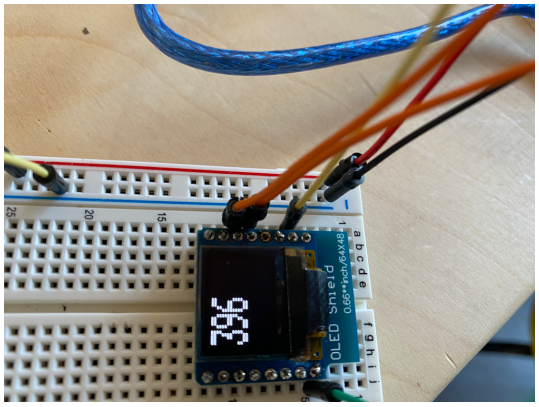
Falls Sie nichts sehen: Positionieren Sie den Cursor so, dass der Text in der linken oberen Ecke Ihres Displays beginnt.

**Frage 2:** Funktioniert Ihr Programm? Wird der Text vollständig auf dem Display dargestellt?

Offensichtlich schneidet das Display die linke obere Ecke ab, und auch der Zeilenumbruch stimmt nicht.

Passen Sie das Programm so an, dass der Text sauber auf mehrere Zeilen verteilt wird.

**Frage 3:** Wie muss der Code in „loop()“ aussehen, damit die Ausgabe vollständig auf dem Display zu sehen ist?




Erweitern Sie Ihr Programm jetzt so, dass es dynamische Werte ausgibt:

- Initialisieren Sie (außerhalb des Loops) eine Variable i vom Typ integer mit dem Wert 0
- Ändern Sie in der Setup-Methode die Textgröße auf 2.
- Im Loop folgen nacheinander diese Schritte:
- Display löschen
- Cursor an die richtige Stelle setzen
- Variable i mit der Methode `display.println` ausgeben
- Text mit `display.display` anzeigen
- 50 ms warten.
- Variable i um 1 erhöhen

**Frage 4:** Was beobachten Sie? Kopieren Sie Ihren kompletten Sketch in die Antwort zu dieser Frage.

## 2) Anbindung von IR-Sensoren

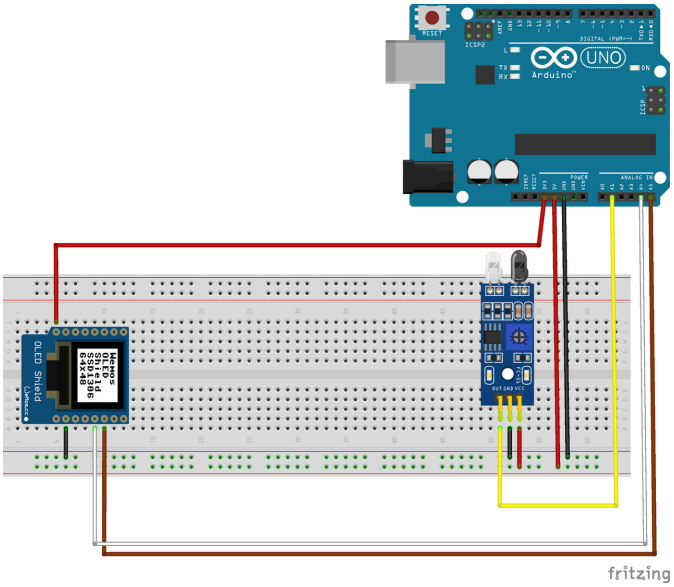
Jetzt erweitern Sie Ihre Anwendung um IR-Sensoren.

### Sie brauchen zusätzlich folgende Komponenten

- IR-Sensor FC-51
- Weitere Kabel

### Aufbau

- Schalten Sie den Arduino wieder stromlos (USB-Kabel abstecken)
- Verbinden Sie den Anschluss GND des Sensors mit der Masseleiste auf Ihrem Breadboard
- Verbinden Sie die 5V Ihres Arduinos mit der entsprechenden Leiste auf dem Breadboard
- Verbinden Sie den VCC Eingang Ihres Sensors mit der Spannungsversorgung am Breadboard
- Verbinden Sie den Ausgang (OUT) Ihres Sensors mit einem analogen Eingang Ihres Arduino (z.B. A1).




Jetzt können Sie den Wert des Sensors mit dem Arduino auslesen. Erweitern Sie dazu das Programm aus der ersten Aufgabe so:

- Statt eines Zählers speichern Sie jetzt den Wert des Eingangs in der Variable i
- Rufen Sie dazu im Loop `analogRead` auf, geben Sie als Parameter den PIN an (z. B. `A1`) und speichern Sie das Ergebnis in der Variable i.
- Geben Sie i anschließend (wie bisher) mit `display.println` auf dem Display aus

**Frage 5:** Funktioniert Ihr Programm? Welchen Wert liefert der Sensor? Bewegen Sie Ihre Hand über den Sensor. Verändert sich der Wert? In welcher Form verändert sich der Wert?

Auf der Rückseite des Sensors sitzt eine kleine Plastikschraube. Drehen Sie sie vorsichtig (!) mit einem passenden Schraubenzieher.

**Frage 6:** Wie wirkt sich die Stellung dieser Schraube auf das Verhalten Ihres Programms aus?

Drehen Sie die Schraube wieder in eine mittlere Position, sodass der Sensor wie gewünscht arbeitet.

Passen Sie das Programm so an, dass der Wert des Sensors digital eingelesen wird.

**Frage 7:** Was wird nun auf dem Display ausgegeben? Welcher Code muss innerhalb der Loop-Methode stehen? Posten Sie den entsprechenden Ausschnitt aus Ihrem Sketch.

Jetzt machen Sie aus dem IR-Sensor einen kontaktlosen Schalter. Das bedeutet: Jedes Mal, wenn die Hand über den Sensor fährt, wird dieser Kontakt einmal (!) gezählt. Solange die Hand über dem Sensor bleibt, zählt das Programm keine weiteren Kontakte. Erst wenn die Hand den Sensor verlässt (und dann wieder über ihn gehalten wird), zählt es einen weiteren Kontakt. Das Display zeigt die Anzahl der kontaktlosen „Klicks“ an.

Gehen Sie so vor:

- Ihr Programm soll eine sogenannte fallende Flanke erkennen (einen Wechsel von 1 auf 0). Das heißt: Sie prüfen mit einer Verzweigung, ob der aktuelle Sensorwert 0 und der Sensorwert des vorherigen Durchlaufs 1 ist.
- Dafür müssen Sie in jedem Loop den aktuellen und den vorherigen Sensorwert kennen.
- Speichern Sie diese Sensorwerte in den Variablen `previous` und `current`. Initialisieren Sie beide (am Anfang Ihres Sketches, außerhalb des Loops) mit 1. Außerdem brauchen Sie eine dritte Variable, `count`, die die Kontakte zählt. Initialisieren Sie sie an derselben Stelle, aber mit dem Wert 0.
- Damit hat der Loop folgende Grundstruktur

```
void loop() {
  display.clearDisplay();
  display.setCursor(32,8);

  current = digitalRead(IR);

  // Zähler hochzählen, wenn neuer Kontakt erkannt.

  display.println(count);
  display.display();
  previous = current;
```

```
}
```

– Ersetzen Sie jetzt den Kommentar durch Programmcode, der eine fallende Flanke erkennt und dann den Zähler um 1 erhöht.

**Frage 8:** Funktioniert Ihr Programm wie gewünscht? Durch welchen Code müssen Sie den Kommentar ersetzen, um die gewünschte Funktionsweise zu erreichen?

Öffnen Sie jetzt das Programm Autodesk Tinkercad (https://www.tinkercad.com) und entwickeln Sie einen Schaltplan für den Anschluss eines Displays (SSD1306) und zweier IR-Sensoren (FC-51). Die exakten Bauteile gibt es dort leider nicht. Verwenden Sie stattdessen das Standard I2C-Display und einen IR-Sensor mit drei PINs. Sie können natürlich auch auf Papier zeichnen, aber mit Tinkercad sieht es einfach besser aus! Lassen Sie sich den Schaltplan von einem der Praktikumsbetreuer freigeben, bevor Sie ihn auf dem Breadboard umsetzen. Bauen Sie die Schaltung mit ausreichend Abstand zwischen den beiden IR-Sensoren auf, damit Sie mit beiden Sensoren unabhängig interagieren können.

Erweitern Sie Ihr Programm jetzt um zwei parallele Zähler, die die (weiterhin berührungslosen) Kontakte unabhängig voneinander zählen. Starten Sie einen kleinen Wettbewerb mit Ihrem*Ihrer Teamkolleg*in: Wer treibt den „eigenen“ Zähler schneller nach oben?

**Frage 9:** Wie viele Variablen brauchen Sie mindestens, damit die zwei Zähler fehlerfrei funktionieren?

## 3) Neue Displays – neue Möglichkeiten

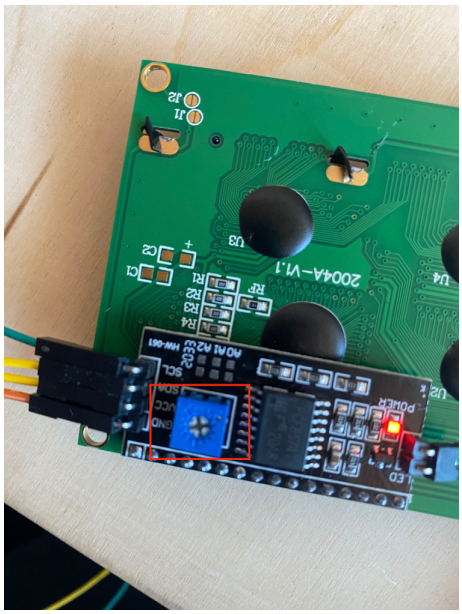
Mit Blick auf den Hackathon lernen Sie jetzt, wie Sie beliebige Hardware mit dem Arduino nutzen, indem Sie die passenden Bibliotheken laden (und einbinden).

Besonders einfach ist es, die Hardware über Funduino (https://funduino.de) zu bestellen. Dort gibt es neben der Hardware auch Anleitungen, wie Sie die Bauteile mit Ihrem Arduino verwenden.

Das Display für diese Aufgabe ist ein vierzeiliges I2C LCD Display für Arduino (https://funduino.de/anleitung-4x20-i²c-lcd-modul). Führen Sie folgende Schritte aus der Online-Anleitung durch:

- Schließen Sie alle vier Leitungen des Displays am Arduino an; beziehen Sie Masse und 5V am besten wieder über die Leisten am Breadboard. Achtung: Dieses **Display benötigt eine Spannung von 5V**.
- Prüfen Sie (wie in der Anleitung erläutert), ob die passende Bibliothek auf Ihrem Rechner schon installiert ist. Falls nicht, laden Sie die Datei LiquidCrystal_I2C direkt aus Moodle herunter. Entpacken Sie sie in Ihrem persönlichen Arduino Ordner (unter Benutzer / Dokumente) im Unterordner `libraries`.

Erstellen Sie einen neuen Sketch, kopieren Sie den Beispielcode hinein und übertragen Sie das Programm auf den Arduino



```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27,20,4);

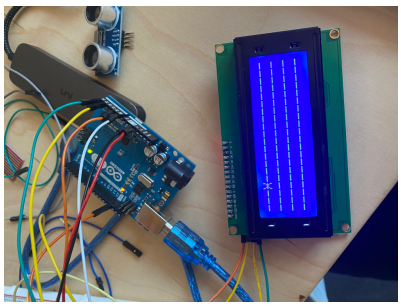
void setup()
{
  lcd.init();
  lcd.init();

  lcd.backlight();
  lcd.setCursor(3,0);
```

```
  lcd.print("Hello World!");
  lcd.setCursor(2,1);
  lcd.print("Arduino UNO R4");
  lcd.setCursor(0,2);
  lcd.print("--");
  lcd.setCursor(2,3);
  lcd.print("TMI Praktikum");
}

void loop() {}
```

– Können Sie keinen Text lesen, ist das Display **vermutlich zu hell oder zu dunkel** eingestellt. Drehen Sie vorsichtig (!) an der Plastikschraube auf der Rückseite des Displays, bis Sie den Text problemlos lesen können.



Jetzt lassen Sie eine kleine Animation auf dem Display laufen. Dazu definieren Sie eine Variable `no` und eine Variable `yes` mit jeweils einem Zeichen Ihrer Wahl. Soll ein `X` über – „fliegen“, wählen Sie als `yes` ein `X` und als `no` einen `-`.

Dann gehen Sie so vor:

– Initialisieren Sie die Variablen `yes` und `no` und einen Zähler (`count`), der bei 0 startet. Außerdem brauchen Sie eine Variable `text` vom Typ `String`; initialisieren Sie sie mit einem leeren String.
– Die Methode `setup` bleibt unverändert.
– Füllen Sie im Loop den `text` mit einer `for`-Schleife mit insgesamt 80 Zeichen (20 Zeichen pro Zeile, 4 Zeilen). Entspricht der Schleifenzähler dem Wert `count`, hängen Sie das Zeichen `yes` an, in allen anderen 79 Fällen das Zeichen `no`.
– Geben Sie den so gefüllten `text` anschließend aus: Positionieren Sie den Cursor in der linken oberen Ecke und nutzen Sie die `println`-Methode des `lcd`-Displays.
– Zum Schluss erhöhen Sie den Counter um eins
– Ist der Counter über 80, beginnt alles wieder von vorn (dafür können Sie z. B. den Modulo-Operator, das `%`-Zeichen, verwenden)
– Bauen Sie am Ende des Loops noch ein `delay` von 10 ms ein.

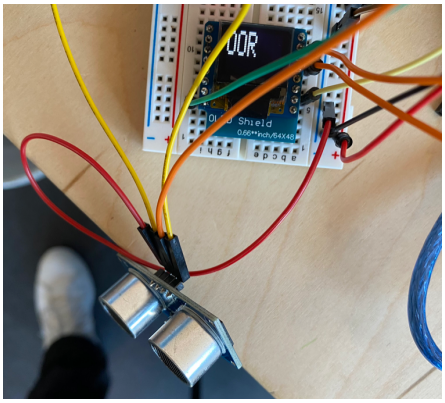
**Frage 10:** Wie „fliegt“ das X jetzt über den Bildschirm? Hätten Sie dieses Verhalten erwartet? Woran könnte das liegen? Wie lässt sich das Problem beheben? Sie müssen das Problem hier nicht lösen; es genügt, die Lösungsidee zu skizzieren.

Jetzt sind Sie gut vorbereitet, um im Hackathon schnell mit neuen Sensoren zu arbeiten!

## 4) Ultraschall

In Aufgabe 2 haben Sie gesehen: Der IR-Sensor eignet sich nicht, wie erhofft, direkt zum Messen von Entfernungen. Um die Position Ihrer Hand halbwegs präzise abzuschätzen, bräuchten Sie ein ganzes Array von IR-Sensoren.

Zum Glück gibt es Ultraschallsensoren. Sie messen Entfernungen deutlich besser.



### Sie brauchen zusätzlich folgende Komponente

- Ultraschallsensor HC-SR04

Ob Sie für diese Aufgabe das erste oder das zweite Display nutzen, ist Ihnen freigestellt. Der Schaltplan unten zeigt die Variante mit dem ersten Display. Achten Sie hier besonders darauf, das **Display mit 3V3 und den Ultraschallsensor mit 5V** zu versorgen.

Egal, welches Display Sie verwenden: Den Ultraschallsensor schließen Sie so an:

- GND an die Masseleiste, VCC an die 5V-Leiste am Breadboard
- Den Trigger (Trig) an einen digitalen Port, z. B. Port 7
- Das Echo ebenfalls an einen digitalen Port, z. B. Port 6

Eine einfache Ansteuerung des Sensors sieht so aus:

```
// je nach Display entsprechende Imports

#define PIN_TRIGGER 7
#define PIN_ECHO    6
#define OLED_RESET 0
unsigned long duration;

void setup()   {
  pinMode(PIN_TRIGGER, OUTPUT);
  pinMode(PIN_ECHO, INPUT);

  // Initialisierung des Displays
}

void loop() {
```

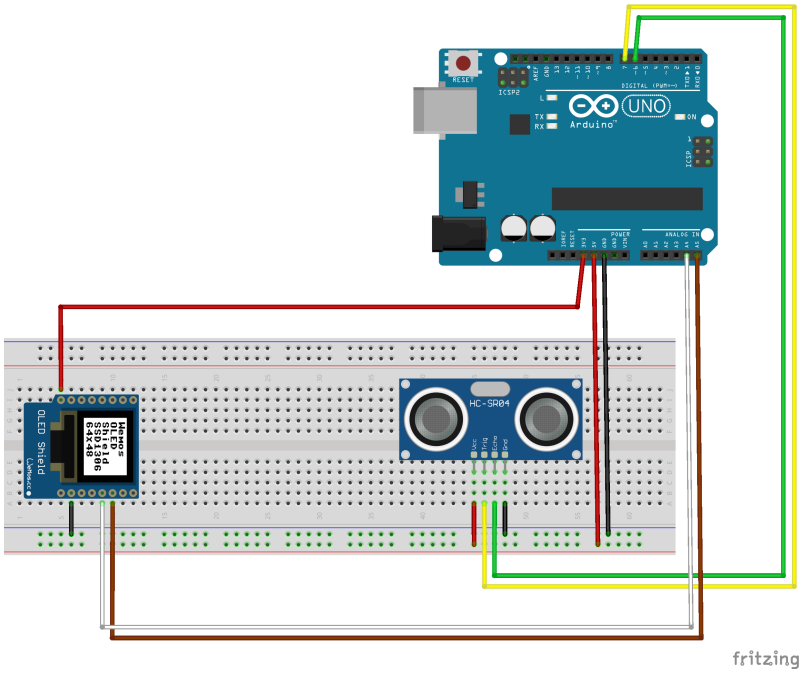
```
  // Display leeren

  digitalWrite(PIN_TRIGGER, LOW);
  delay(5);
  digitalWrite(PIN_TRIGGER, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIGGER, LOW);

  duration = pulseIn(PIN_ECHO, HIGH);

  // Variable duration auf dem Display ausgeben

  delay(100);
}
```




Versuchen Sie, das Programm nachzuvollziehen:

- Der `Trigger` dient als Output, das Echo als Input
- Am `Trigger` liegt kurz (d. h. 10 µs) ein `HIGH`-Pegel an; daraufhin sendet der Sensor ein Schallsignal
- Dann wird das Signal wieder ausgeschaltet, und `pulseIn` liest am Pin `Echo` die Laufzeit aus

**Frage 11:** Bewegen Sie sich jetzt langsam vom Sensor weg. Wie verändert sich die Laufzeit? In welchem Bereich erkennt der Sensor (ungefähr) hinreichend große Objekte (z. B. eine Wand)?

Wie viele Mikrosekunden der Schall zum Objekt braucht, ist allerdings nicht besonders aufschlussreich. Viel hilfreicher wäre die Entfernung! Kein Problem, die Schallgeschwindigkeit ist ja erfreulicherweise bekannt:

- Die Schallgeschwindigkeit beträgt 1235,5 km/h (in trockener Luft)
- Das entspricht 343,2 m/s
- Das sind 34.320 cm/s, aber `pulseIn` misst in µs (eine Millionstel Sekunde, 0,000001 Sekunden).
- Multiplizieren Sie also Ihren Messwert (in µs) mit der Schallgeschwindigkeit (in cm/µs), erhalten Sie die Entfernung des Objekts in cm.

Passen Sie den Code entsprechend an und geben Sie die Entfernung aus. Testen Sie empirisch, ob Ihre Berechnung stimmt. Vermutlich ist Ihnen dabei ein kleiner Denkfehler unterlaufen. Denken Sie noch einmal darüber nach, wie der Sensor funktioniert, und korrigieren Sie Ihre Berechnung.

**Frage 12:** Wie lautet die korrekte Formel für die Berechnung der Distanz in cm?

Geben Sie die Distanz jetzt nur aus, wenn sie zwischen 0 und der maximalen Reichweite liegt. Liegt der Messwert außerhalb dieses Bereichs, geben Sie stattdessen `OOR` aus (für „out of range“).

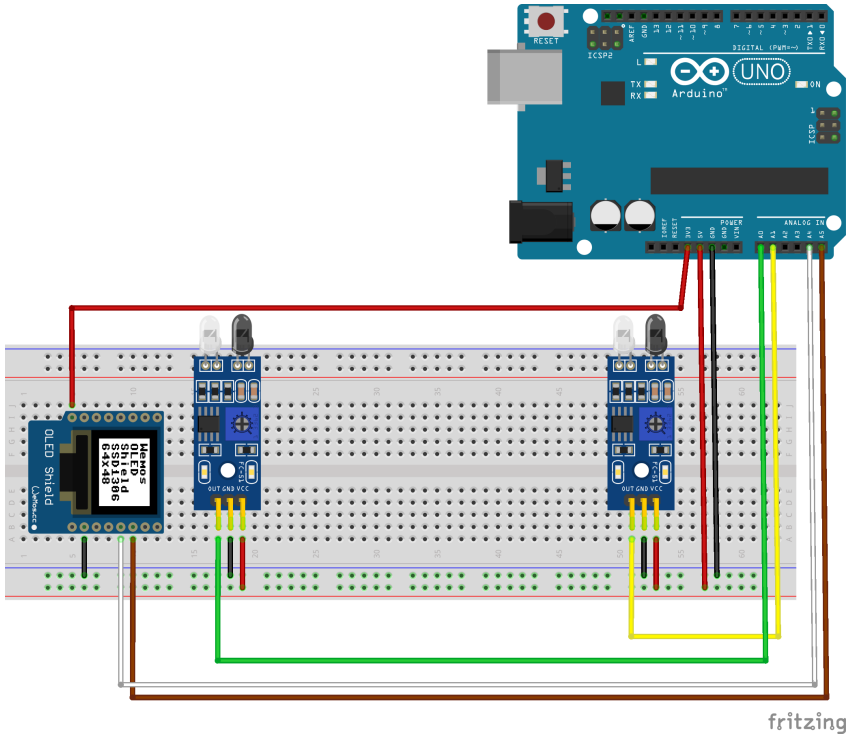
**Frage 13:** Wie sieht die finale Version des Codes innerhalb der Loop-Methode aus?

## 5) Kreativ werden

Wenn Sie noch Zeit haben, nutzen Sie die bisherigen Sensoren für eine kleine, kreative Anwendung. Zum Beispiel können Sie die beiden IR-Sensoren als kontaktlose Buttons für ein kleines Quiz verwenden.

Grundsätzlich können Sie Ihrer Kreativität aber freien Lauf lassen. Statt eines Quizzes können Sie z. B. auch auf dem Beispielprogramm zum SSD1306-Display aufsetzen (zu finden unter „`Datei -> Beispiele -> Beispiele aus eigenen Bibliotheken -> Adafuit SSD 1306 -> ssd1306_128x64_i2c`“) und die Animation ggf. an das kleinere Display anpassen, indem Sie wieder den Offset von `32, 8` berücksichtigen.

Wenn Sie zum Abschluss das Quiz bauen möchten, gehen Sie so vor: Verkabeln Sie eines der beiden Displays und die beiden IR-Sensoren:




Öffnen Sie anschließend das Beispiel aus Moodle (`IR_Kollisionen_quiz.ino`). Das Quiz speichert die Fragen in einem zweidimensionalen Array:

```
String fragen[3][2] = { { "Ist heute", "Montag?" }, { "Ist heute", "Dienstag?" }, { "Ist heute",
"Mittwoch?" } };
```

In diesem Beispiel besteht jede Frage aus zwei Zeilen (sind Ihre Fragen länger, lässt sich das problemlos anpassen), und es gibt insgesamt drei Fragen. Fügen Sie im ersten Schritt insgesamt 10 Fragen ein. Denken Sie daran, die erste Dimension des Arrays auf `10` zu erhöhen. Brauchen Sie mehr als zwei Zeilen, passen Sie auch die zweite Dimension an.

Die Antworten sind so codiert:

```
int antworten[3] = { 0, 0, 1 };
```

Die Länge des Arrays Antworten entspricht natürlich der ersten Dimension des Fragen-Arrays, in diesem Beispiel also zunächst drei. Ist eine Frage falsch, ist die Antwort 0; ist sie wahr, lautet die Antwort 1.

Die Methode `punktestand` gibt nach jeder Antwort den Zwischenpunktestand aus und am Ende des Quiz den Endpunktestand.

So gibt ein Spiel die Fragen der `runde` aus:

```
display.setCursor(32,8);
display.println(fragen[runde][0]);
display.setCursor(32,16);
display.println(fragen[runde][1]);
display.setCursor(32,24);
display.println("ja / nein");
```

Falls Ihre Fragen aus drei Zeilen bestehen, dann ersetzen Sie die letzte Zeile („ja / nein“) durch die Ausgabe Ihrer dritten Zeile.

Wer richtige Fragen mit ja und falsche mit nein beantwortet, bekommt einen Punkt. Je nach Verkabelung müssen Sie die Logik ggf. invertieren, damit der Sensor für ja links und der Sensor für nein rechts liegt:

```
if ((prev_ir1 == 1) && (ir1 == 0)) {
  // ja
  if (antworten[runde] == 1) {
    punkte++;
  }
  (...)
} else if ((prev_ir2 == 1) && (ir2 == 0)) {
  // nein
  if (antworten[runde] == 0) {
    punkte++;
  }
  (...)
}
```

Weil Sie mehr als drei Fragen haben, müssen Sie zuletzt noch den Spielabbruch anpassen.

**Frage 14:** Posten Sie als Antwort auf diese letzte Frage den Inhalt Ihres „kreativen“ Arduino Sketches.